CS543 • WEB SERVICES

ASSIGNMENT 6 OF 12

Honest Responses: Status Codes, Errors & Validation

Group ID: 17

TEAM MEMBERS

Student Name	Roll Number
Anurag Tiwari	20251651028
Saurabh Singh	20251651089
Mohd Rashid	20251651057
Niraj Kumar Gupta	20251651066

Part A — Status Codes

AlgoViz API status mapping with Spring Boot code

A1. Short theory

HTTP status codes tell the frontend whether an API request succeeded or failed. The response status is the result: do not return 200 OK for a failed request with a body such as {"ok":false}. For a new visualization, use 201 Created and a Location header. Use 4xx for caller/request problems and 5xx for server-side failures.

A2. Endpoint status map

These are proposed paths based on the AlgoViz project description. Rename paths to match the actual controller routes in your code.

Endpoint	Success	Common failures
GET /api/algorithms	200 OK	401/403 if protected; 500 unexpected error.
POST /api/visualizations	201 Created	400 malformed JSON; 401 unauthenticated; 409 idempotency conflict; 422 invalid algorithm/input; 429 rate limit; 500/503 server fault.
GET /api/visualizations/{id}	200 OK	401 unauthenticated; 404 unknown/inaccessible run; 500 server fault.
GET /api/history	200 OK	401 unauthenticated; 500 server fault.
DELETE /api/history/{id}	204 No Content	401 unauthenticated; 404 missing/inaccessible record; 500 server fault.

A3. Spring Boot example: create visualization

```
@PostMapping("/api/visualizations")
public ResponseEntity<VisualizationResponse> create(
@Valid @RequestBody VisualizationRequest request) {
VisualizationResponse result = visualizationService.create(request);
URI location = URI.create("/api/visualizations/" + result.id());
return ResponseEntity.created(location).body(result);
}
```

A4. Example DTOs

```
public record VisualizationRequest(
@NotBlank String algorithm,
@NotEmpty List<Integer> input) {}

public record VisualizationResponse(
String id,
String algorithm,
List<Integer> input,
String status) {}
```

A5. Retry guidance

```
// For 429/503 responses, include Retry-After.
// Example: return ResponseEntity.status(503)
// .header(HttpHeaders.RETRY_AFTER, "5")
// .body(problemBody);
```

Notes / implementation caution

A successful POST returns 201 and Location. If your backend queues work, use a documented 202 Accepted contract instead. Do not copy both approaches into the same endpoint without deciding which one your service uses.

Part B — One Error Shape

Problem Details, validation errors, and safe exception handling

B1. Short theory

A shared error format makes it easier for React to show clear messages. The response uses type, title, status, and detail. For validation, add errors[] with the name of each invalid field and a reason. Use Content-Type: application/problem+json and never expose stack traces or database messages.

B1.5 Error type catalogue

Error type	Status	When it happens
unsupported-algorithm	422	Algorithm is not supported.
invalid-input	422	Input values or array are invalid.
visualization-not-found	404	Requested run is missing or inaccessible.
illegal-transition	409	Action is not allowed in the current state.
idempotency-key-conflict	409	Same idempotency key used with a different body.
rate-limit-exceeded	429	Too many requests.
service-unavailable	503	Temporary service outage.
internal-error	500	Unexpected server-side problem.

B2. Problem Details model

```
public record ApiProblem(
String type,
String title,
int status,
String detail,
List<FieldError> errors) {}

public record FieldError(String field, String reason) {}
```

B3. Central exception handler

```
@RestControllerAdvice
public class ApiExceptionHandler {
@ExceptionHandler(MethodArgumentNotValidException.class)
ResponseEntity<ApiProblem> validation(
MethodArgumentNotValidException ex) {
List<FieldError> errors = ex.getBindingResult()
.getFieldErrors().stream()
.map(e -> new FieldError(
e.getField(), e.getDefaultMessage()))
.toList();

ApiProblem body = new ApiProblem(
"https://algoviz.example/problems/invalid-input",
"Invalid visualization input", 422,
"Correct the fields listed in errors.", errors);

return ResponseEntity.unprocessableEntity()
.contentType(MediaType.valueOf(
"application/problem+json"))
.body(body);
}

@ExceptionHandler(Exception.class)
ResponseEntity<ApiProblem> unexpected(Exception ex) {
// Log ex with a request/correlation ID on the server.
ApiProblem body = new ApiProblem(
"https://algoviz.example/problems/internal-error",
"Internal server error", 500,
"The request could not be completed.", List.of());
return ResponseEntity.status(500)
.contentType(MediaType.valueOf(
"application/problem+json"))
.body(body);
}
}
```

B4. Example 422 response

```
{
"type": "https://algoviz.example/problems/invalid-input",
"title": "Invalid visualization input",
"status": 422,
"detail": "Correct the fields listed in errors.",
"errors": [
{"field": "algorithm", "reason": "Unsupported algorithm."},
{"field": "input", "reason": "Input must not be empty."}
]
}
```

Notes / implementation caution

Use the same error contract for known failures across endpoints. In production, avoid a broad exception handler that accidentally hides useful distinctions: map known exceptions to suitable 4xx statuses and keep unknown exceptions as 500.

Part C — Validation and Content Formats

Validate AlgoViz input before algorithm execution

C1. Short theory

Validation must happen before running an algorithm or saving history. Collect all field errors and return one 422 response. JSON is the default response format; XML may be returned only if the service implements it. If the requested Accept format cannot be produced, return 406 Not Acceptable.

C6. Validation summary

Rule	Expected handling
Missing algorithm	422 with algorithm field error.
Unsupported algorithm	422 with supported-algorithm error.
Empty input	422 with input field error.
Input over configured limit	422 with input-size error.
Malformed JSON	400 Bad Request.
Unsupported Accept type	406 Not Acceptable.

C2. Request validation DTO

```
public record VisualizationRequest(
@NotBlank String algorithm,
@NotEmpty List<@NotNull Integer> input) {}
```

C3. Validate algorithm and input before processing

```
private static final Set<String> SUPPORTED = Set.of(
"BUBBLE_SORT", "SELECTION_SORT", "INSERTION_SORT",
"LINEAR_SEARCH", "BINARY_SEARCH");

public void validateRequest(VisualizationRequest req) {
List<String> errors = new ArrayList<>();

if (req.algorithm() == null ||
!SUPPORTED.contains(req.algorithm())) {
errors.add("algorithm must be supported");
}
if (req.input() == null || req.input().isEmpty()) {
errors.add("input must not be empty");
}
if (req.input() != null && req.input().size() > 500) {
errors.add("input must contain at most 500 values");
}
if (!errors.isEmpty()) {
throw new InvalidVisualizationInputException(errors);
}
// Only call the Java algorithm layer after validation passes.
}
```

C4. Content-Type and Accept

```
// JSON response example
return ResponseEntity.ok()
.contentType(MediaType.APPLICATION_JSON)
.body(response);

// Problem Details errors should use:
// application/problem+json; charset=utf-8

// If XML is not implemented and the client requires XML,
// return 406 Not Acceptable rather than sending JSON as XML.
```

C5. XML negotiation option

```
// If XML support is implemented, configure an XML message
// converter and a response representation for the endpoint.
// Then test with:
// curl -i -H "Accept: application/xml" \
// http://localhost:8080/api/algorithms
//
// Do not claim XML support until this test returns XML.
```

Notes / implementation caution

The exact supported algorithms and maximum input length should match the current AlgoViz implementation. The sample limit of 500 is an illustrative project rule.

Part D — Curl Tests and Submission

Run the endpoints and capture real status codes and bodies

D1. Short theory

curl -i prints the HTTP status line and response headers as well as the body. Use it to verify the status, Location, Content-Type, and Problem Details body. The examples below are templates; replace the endpoint paths and authentication requirements with your actual project settings.

D2. Successful create (happy path)

```
curl -i -X POST http://localhost:8080/api/visualizations \
-H "Content-Type: application/json" \
-H "Accept: application/json" \
-H "Idempotency-Key: run-06-001" \
-d '{"algorithm":"BUBBLE_SORT","input":[5,2,4,1]}'

// Expected contract: 201 Created and Location:
// /api/visualizations/{id}
// Use 202 instead only if your API queues the job.
```

D3. Invalid request

```
curl -i -X POST http://localhost:8080/api/visualizations \
-H "Content-Type: application/json" \
-H "Accept: application/problem+json" \
-H "Idempotency-Key: run-06-002" \
-d '{"algorithm":"UNKNOWN","input":[]}'

// Expected: 422 with errors[] listing algorithm and input.
```

D4. Unknown visualization ID

```
curl -i http://localhost:8080/api/visualizations/not-a-real-id

// Expected: 404 with a Problem Details body.
```

D5. XML negotiation

```
curl -i -H "Accept: application/xml" \
http://localhost:8080/api/algorithms

// Expected: XML if implemented; otherwise 406 Not Acceptable.
```

D6. Rate-limit / unavailable tests

```
// In a test environment, trigger configured rate limiting
// and verify:
// HTTP/1.1 429 Too Many Requests
// Retry-After: 5
// Content-Type: application/problem+json

// For temporary unavailability:
// HTTP/1.1 503 Service Unavailable
// Retry-After: 5
// Content-Type: application/problem+json
```

D7. Regression test for idempotency

```
// Send the exact same POST twice using the same key and body.
// Confirm both responses point to the same logical visualization
// and that the database/history contains only one run.
// A timeout does not prove the server failed: retry with the
// same key and body, not a new key.
```

Notes / implementation caution

Submission checklist: updated service source; updated openapi.yaml with endpoint status codes and Problem Details schema; transcript.txt containing actual curl output for success and failure cases; NOTES.md answering the assignment questions. Add student/team names and roll numbers to the first page.